

An Agentic Retrieval-Augmented Generation Framework for Multi-Hop Question Answering Using Multi-Agent Collaboration

Jishnu K^{a,*}, Balaji V^a, Karthikeyan P^a, Ashwini Kumar Mathur^a

^a*School of Computer Science and Engineering, RV University, Bengaluru, India*

Abstract

Conventional Retrieval-Augmented Generation (RAG) systems typically rely on a single retrieval step and lack query decomposition, iterative evidence retrieval, and explicit response verification. These limitations make them less effective for multi-hop question answering, which requires connecting evidence from multiple passages to derive accurate answers. This paper presents an Agentic RAG framework that divides question answering into four collaborative stages: planning, evidence retrieval, reasoning, and verification implemented using LangGraph. The Planner Agent decomposes questions into targeted sub-queries, the Evidence Retrieval Agent searches a ChromaDB-based vector index, the Reasoner Agent synthesizes evidence and generates answers, and the Verifier Agent independently re-retrieves evidence to validate each claim and compute an Evidence Sufficiency Score (ESS). Unlike Self-RAG, which relies on model-generated reflection tokens, the proposed verifier operates as a deterministic module. Evaluated on 120 multi-hop questions from HotpotQA, the proposed system achieves an Exact Match of 59.17% and an F1 score of 68.87%, representing improvements of 29.17% and 33.28%, respectively, over the RAG baseline, with an average ESS of 0.82.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: retrieval-augmented generation; multi-agent systems; multi-hop question answering; large language models; hallucination detection; answer verification; LangGraph

1. Introduction

Even though LLMs have worked effectively in question answering, the knowledge base of these models is Fixed to their training data. In cases where the answer to the question lies outside the scope of the training data, the output can be hallucinated or error [1]. Retrieval-Augmented Generation (RAG) addresses this limitation by conditioning generation on documents retrieved from an external corpus at the time of answering, rather than relying completely on parametric knowledge. Nonetheless, most existing single-pass RAG models perform retrieval from a fixed set of passages that is computed using just the raw query and without any decompositions. Such a strategy does not work well for *multi-hop* questions, where multiple inference steps have to be performed based on evidence from two or more documents to get to the answer for instance, figuring out an intermediary entity from one document and then the answer from another. A single un-decomposed retrieval step may not return all the necessary evidence for such

* Corresponding author

E-mail address: jishnuk7676@gmail.com (Jishnu K).

1877-0509 © 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

questions, even when the evidence is correct, there is no validation of provided answer. Two research directions have been pursued recently in order to solve these issues. In one direction, improvements have been made to the search and reasoning process using techniques such as query decomposition and chain-of-thought reasoning. The other approach involves adding self-reflection/self-correction into generation. [2]. The Self-RAG system makes use of just one model to produce tokens of reflection that decide if and how the model's output needs to be retrieved and reviewed. The problem with this technique is that the evaluation of the correctness of the generation done by the system depends upon the same model. The major contributions of this work are summarized as follows:

1. A multi-hop Question Answering system with an agentic RAG architecture which breaks down the question answering procedure into planning, retrieving, reasoning, and verifying components using collaborative multi-agent orchestration.
2. A deterministic claim level Verifier Agent which autonomously retrieves evidence for every atomic claim in the answer that is generated, It produces an interpretable Evidence Sufficiency Score, different from previous self-reflective models which use self-critique from a singular model.
3. An empirical evaluation on HotpotQA compared to a single shot RAG model and pipeline variations thereof, in order to study the individual effect of query decomposition, evidence generation, and verification on the accuracy of the answer generated.
4. A study of the trade-off between latency and accuracy caused by verification through multi-agent collaboration and failure cases such as entity confusion during autonomous evidence retrieval and noise in the dataset annotations.

The remaining sections of this paper are structured as follows. Section 2 summarizes previous work on retrieval-augmented generation, agentic reasoning, and self-corrective retrieval. Section 3 presents the architecture of the proposed system. Section 4 details the experimental setting and includes the dataset, configurations, and metrics used. Section 5 provides and discusses the results of the experiments. Section 6 summarizes this paper and future directions.

2. Related Work

2.1. Retrieval-Augmented Generation

The idea of RAG [1] was developed in order to condition the language model generation on the retrieved documents from an external knowledge source and thus overcome the issue of the static-knowledge bottleneck of parametric-only methods. Karpukhin et al. [3] suggested Dense Passage Retrieval (DPR) in which dense bi-encoders are used for passage retrieval instead of using sparse lexical matching, and Gao et al. [4] introduced HyDE where the hypothetical answer document is generated in order to improve dense retrieval in the zero-shot setting. The above-mentioned works provide the basis for the retrieval component that is used in our research; however, none of them explicitly deals with multi-hop query decomposition, coordinated agentic reasoning, and answer verification individually. Recently, the ICMLDE community has been considering the potential of incorporating RAG with small models in agentic chatbots [5]. Nevertheless, the proposed approach takes an additional step of separating planning, evidence retrieval, reasoning, and claim verification into individual components.

2.2. Reasoning and Agentic Prompting Strategies

In addition to retrieval, a distinct line of research has sought to improve LLM reasoning abilities. The Chain-of-Thought prompting technique [6] demonstrated that reasoning about intermediate steps can help in solving multi-step problems; Self-Consistency [7] extended this idea by generating multiple reasoning traces and combining them, and the Tree of Thoughts [8] method generalized reasoning through a tree of reasoning states. ReAct [9] combined reasoning traces with concrete actions, like search queries, that allowed the model to switch back and forth between reasoning and gathering more data in the same trace. Reflexion [10] brought verbal self-reinforcement to the table, whereby an agent could reason about its own failed attempt in natural language and use that reasoning to retry the task. Toolformer [11] has shown that models can learn, in a self-supervised fashion, when and how to use external tools.

All these methods form the basis of the agentic-reasoning paradigm that our Planner and Reasoner agents follow, but they do not combine step-wise reasoning with a deterministic verification phase of the kind proposed here.

2.3. Multi-Agent and Tool-Using Systems

In addition to single agent reasoning approaches, there have been frameworks that combine multiple specialized components. WebGPT [12] combined the browsing model with human feedback to conduct question answering through live web search. HuggingGPT [13] employed an LLM as a controller that split up tasks and allocated them to specialized models available on Hugging Face. AutoGen [14] offered a general multi-agent framework where agents played different roles and worked together to accomplish a certain task. Modular architecture designs for language processing have also been developed that separate semantic comprehension, contextual integration, and generation into separate modules [15]. The above frameworks show the feasibility of task decomposition among role-specialized agents, which led us to employ four such agents; nevertheless, none of them was designed for retrieval-based QA, nor did any include a claim level verification process.

2.4. Self-Correcting and Adaptive Retrieval

The closest prior work to this includes the area of self-reflective and corrective RAG. The first of these is Self-RAG, which we consider our base paper, where a language model is trained to produce reflection tokens that determine whether and how to retrieve and evaluate its own generated output based on relevancy and support, thereby integrating the two concepts in one model. The second, CRAG [16], trains a small evaluator model to determine the quality of the retrieved document and, if deemed inadequate, trigger correction, for instance, by performing web searches. Adaptive-RAG [17] adapts its retrieval strategy based on query complexity. FLARE [18] uses active retrieval, starts a new retrieval process whenever the model's confidence in predicting its next token drops below a certain threshold value, instead of doing just one retrieval at the beginning.

2.5. Multi-Hop Question Answering

Regarding multi-hop QA in particular, IRCOT [20] interweaves CoT generation with retrieval, where each intermediate reasoning sentence is used for retrieval as well. RAPTOR [21] builds a hierarchical tree of recursively generated summaries of document clusters to provide retrieval capabilities at different levels of abstraction, and RankRAG [22] integrates context ranking and answer generation in an instruction-tuned model. Finally, GraphRAG [23] constructs an entity-relation graph over the corpus to enable query-oriented summarization of document collections. Such methods mainly contribute to the question of *how* to structure or rank the evidence before generation; our method complements such works by addressing *verifying* the answer after generation. The role of structured relationships between pieces of information has also been examined in studies of more complicated document comprehension. For instance, the Rhetorical Role Relatedness model applies both rhetorical roles and semantic relatedness in organizing information for the purpose of generating case briefs.

3. Methodology

3.1. Overview

The proposed framework breaks down multi-hop question answering into four agents working together at four different stages including query decomposition, evidence retrieval, evidence synthesis and answer drafting, and finally claim level verification. Unlike one-shot pipelines for RAG where passages are retrieved once and the generation is done conditioned on them, in our framework we have an extra stage of verification, where each individual claim of the generated response is verified with respect to independent evidence retrieval. The architecture of the end-to-end pipeline is shown in Fig. 1.

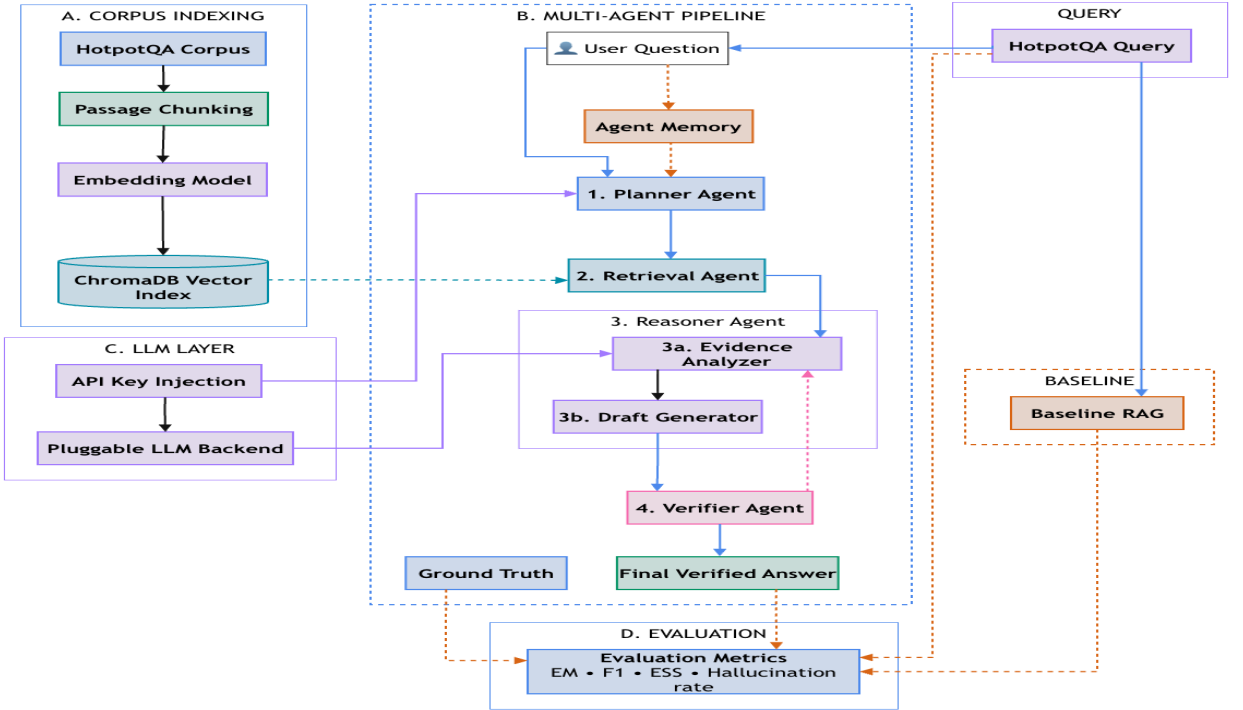


Fig. 1. Overall architecture of the proposed Agentic Retrieval-Augmented Generation framework.

3.2. Corpus Indexing

The corpus for the retrieval step has been developed using the supporting paragraphs from the HotpotQA dataset. Document-level NLP pipelines have also investigated the techniques like text extraction, text segmentation, semantic modeling, and question answering using technical manuals as input sources, underscoring the significance of structured document processing to solve subsequent QA tasks [25]. In our approach, these paragraphs are divided into passage chunks. Dense vectors are generated for each passage chunk by employing the Sentence-Transformer embedding model `all-MiniLM-L6-v2`, and the embeddings of these passages with additional metadata (document title, paragraph number, and chunk number) are saved in ChromaDB.

3.3. Planner Agent

The Planner Agent analyzes the question and decomposes the question into sub-questions corresponding to individual objectives. The prior work [26] has explored the problem of question comprehension, correction, and question difficulty classification utilizing transformers and neural networks emphasizing the need for proper parsing of question structures before any further processings. In the case of bridge and comparison questions in the dataset of HotpotQA, such parsing will help to differentiate between the reasoning hops which cannot be differentiated by a one-shot retrieval agent, such as distinguishing the sub-question that retrieves the intermediary entity from the sub-question that retrieves the answer based on the intermediary entity.

Algorithm 1 End-to-End Agentic RAG Query Processing**Require:** User query Q , vector database D **Ensure:** Verified response A

- 1: Receive multi-hop query Q
- 2: Analyze Q using the Planner Agent
- 3: Decompose Q into retrieval-oriented sub-queries Q'
- 4: Retrieve candidate evidence E from D using Q'
- 5: Remove duplicate evidence from E
- 6: Rank retrieved evidence according to relevance
- 7: Select the highest-ranked evidence set E^*
- 8: Analyze E^* using the Reasoner Agent
- 9: Synthesize atomic key facts from E^*
- 10: Generate draft response A_d using the selected LLM
- 11: Extract atomic claims C from A_d
- 12: Independently retrieve evidence for each claim
- 13: Compare claims against the retrieved evidence
- 14: Compute Evidence Sufficiency Score (ESS)
- 15: **if** all critical claims are sufficiently supported **then**
- 16: Generate the verified response with supporting citations
- 17: $A \leftarrow A_d$
- 18: **else**
- 19: Identify unsupported claims
- 20: Return unsupported claims to the reasoning stage
- 21: Regenerate the response using validated evidence
- 22: **end if**
- 23: Attach supporting citations and ESS to A
- 24: **return** A

The overall execution of the proposed framework is formalized in Algorithm 1. The algorithm describes the complete flow from the initial multi-hop query through query decomposition, evidence retrieval, evidence synthesis, claim-level verification, and generation of the final verified response.

3.4. Evidence Retrieval Agent

Each sub-query generated by the Planner is employed in the process of semantic search within the vector index. The passages retrieved are then deduplicated and ranked according to their relevance, and the collection of passages is combined into an evidence bundle for the Reasoner Agent. Algorithm 2 summarizes the evidence retrieval process, including semantic search, duplicate removal, relevance scoring, and evidence ranking.

3.5. Reasoner Agent

The Reasoner Agent compiles the collected evidence package into atomic key facts, sorting out any interdependencies or possible redundancies between the passages collected from different sources. Prior to finalizing the draft answer, the Reasoner Agent conducts an internal self-critique phase, in which it scrutinizes its initial reasoning process for any logical flaws.

3.6. Verifier Agent

The Verifier Agent conducts a deterministic, after-the-fact verification of the Reasoner's draft response. First, the Verifier Agent selects a set of atomic assertions from the draft response, conditional on the initial question as well as the draft response. The Verifier Agent then poses an independent search query against the ChromaDB index, in order

Algorithm 2 Evidence Retrieval and Ranking**Require:** Retrieval queries Q' , vector database D **Ensure:** Ranked evidence set E^*

```

1: Initialize candidate evidence set  $E \leftarrow \emptyset$ 
2: for each query  $q \in Q'$  do
3:   Generate query embedding  $v_q$ 
4:   Retrieve top- $k$  candidate chunks from  $D$ 
5:   Add retrieved chunks to  $E$ 
6: end for
7: Remove duplicate chunks from  $E$ 
8: for each evidence chunk  $e \in E$  do
9:   Compute retrieval similarity  $S_r(e)$ 
10:  Obtain planner confidence  $S_p(e)$ 
11:  Compute ranking score:  $S(e) \leftarrow 0.6S_r(e) + 0.4S_p(e)$ 
12: end for
13: Sort  $E$  in descending order of  $S(e)$ 
14: Select highest-ranked evidence chunks
15: return  $E^*$ 

```

Algorithm 3 Evidence-Based Response Verification**Require:** User query Q , draft response A_d , evidence set E **Ensure:** Verified response A

```

1: Extract factual claims  $C = \{c_1, c_2, \dots, c_n\}$  from  $A_d$ 
2: Initialize supported-claim count  $N_s \leftarrow 0$ 
3: for each claim  $c_i \in C$  do
4:   if  $c_i$  is supported by  $E$  then
5:     Mark  $c_i$  as supported and update  $N_s \leftarrow N_s + 1$ 
6:   else
7:     Mark  $c_i$  as unsupported
8:   end if
9: end for
10: Compute Evidence Sufficiency Score:  $ESS \leftarrow \frac{N_s}{|C|}$ 
11: if all critical claims are supported then
12:   Attach supporting citations and set  $A \leftarrow A_d$ 
13: else
14:   Identify unsupported claims
15:   Correct unsupported claims using available evidence
16:   Regenerate verified response  $A$ 
17: end if
18: return  $A$ 

```

to assess the support of each individual claim independently of the context that is generated within the corpus. The Evidence Sufficiency Score (ESS) is calculated as:

$$ESS = \frac{N_{\text{sup}}}{N_{\text{total}}} \quad (1)$$

Where N_{sup} refers to the number of atomic claims confirmed independently using the evidence retrieved and N_{total} refers to the total number of atomic claims obtained from the draft answer. Through this Claims that does not match with the independently retrieved evidence will be identified; also, the Verifier will create source citations at the source level for every supported claim, which will be added along with the ESS score to the final answer.

Verification process for validation of generated claim against the retrieved evidence along with calculation of the Evidence Sufficiency Score (ESS) is shown in Algorithm 3. Claims with insufficient evidence are sent back to the reasoning process.

3.7. Pluggable LLM Backend

Reasoner and Verifier are independent of any one LLM vendor. API keys are dynamically passed in and are valid only during the life cycle of the request itself, thereby enabling switching of backends between different vendors such as OpenAI, Anthropic, and Groq or locally running models such as using Ollama, without any modifications to the behavior of the agents. In all experiments reported here, llama-3.1-8b-instant from Groq is used as the underlying LLM in the Reasoner, Verifier, Planner, and Draft Generator agents.

4. Experimental Setup and Evaluation metrics

4.1. Dataset

We tested our proposed model on the HotpotQA dataset¹. It is a benchmark for multi-hop question answering with examples sourced from Wikipedia that require combining evidence from two or more supporting paragraphs. To circumvent computing restrictions and API quota limits, our experiments were run on a stratified sampling of $N = 120$ questions from the HotpotQA distractor, including both *bridge* and *comparison* reasoning varieties. Our retrieval set was created by splitting up the corresponding supporting paragraphs from Wikipedia based on a 1000 character passage size and 200 character overlap using LangChain's RecursiveCharacterTextSplitter.

4.2. Configurations Compared

We compare four configurations on the same set of 120 questions: Baseline RAG, which performs single-shot retrieval without decomposition or verification; Stage 1 (Decomposition), which adds query decomposition and sub-query retrieval; Stage 2 (Reasoner), which further introduces evidence synthesis and self-critique and Stage 3 (Full System), which adds the deterministic Verifier Agent to complete the proposed pipeline.

4.3. Evaluation Metrics

The evaluation is performed using standard HotpotQA metrics including EM and F1 score computed for normalized answer spans (in lowercase, without articles and punctuation marks); Supporting Fact F1 score (Sp F1) metric for the set of the supporting facts pointed out by the model against the gold standard set of support facts and Joint EM/F1 metric for instance harmonized metric for the answer span and supporting fact metrics. Evaluation of automatically generated open answers is another aspect because generated open answers may differ in quality and correctness [27]. Also, mean Evidence Sufficiency Score (ESS) is measured using Eq. 1.

5. Results and Discussion

5.1. Overall Performance

Table 1 reports Exact Match (EM), F1, Supporting-Fact F1, Joint EM/F1, latency, and mean Evidence Sufficiency Score (ESS) for the four configurations described in Section 4, evaluated on an identical set of $N = 120$ HotpotQA multi-hop questions. For the system, which is Stage 3 we get an accuracy of 59.17 % and a score of 68.87 percent. This is an improvement over the baseline system. We are talking about a gain of 29.17% points and 33.28% points. Nevertheless this improvement is not the same across the system. Stage 1 by itself does not do better than the baseline

¹ https://huggingface.co/datasets/hotpotqa/hotpot_qa

system. Indeed it does a worse. This tells us that breaking down a query into parts does not really help us get better answers. It can even add some mistakes when we try to answer these queries on their own without putting them together again. The big improvement actually happens at Stage 2. This is when the Reasoner Agent takes the information we found and breaks it down into key facts before we generate an answer. Then Stage 3 makes it even better, with the Verifier Agents process of checking and correcting our answers. The complete system, which is Stage 3 really shows us how these different stages work together to get better results

Table 1. Performance comparison across architectural configurations on HotpotQA (N = 120)

Configuration	EM (%)	F1 (%)	Sp F1 (%)	Joint F1 (%)	Latency (s)	ESS
Baseline RAG	30.00	35.59	0.00	0.00	0.39	–
Stage 1 (Decomp.)	29.17	34.76	0.00	0.00	0.93	–
Stage 2 (Reasoner)	45.00	54.51	0.00	0.00	4.43	–
Stage 3 (Full)	59.17	68.87	13.89	9.59	36.84*	0.82

5.2. Latency Analysis

A latency cost is associated with the end-to-end pipeline in relation to the single pass, considering the five calls to the LLM (Planner, Reasoner, self-critique, Answer Generator, and Claim Extractor) and the two passes for vector search that it makes. In order to report on this correctly, it is necessary to make a distinction between the wall clock evaluation time and the actual overhead cost of the system. The unfiltered mean latency over all the 120 questions is 157.87 s, a value highly affected by a handful of outliers (maximum 2,694 s), which are the consequence of exponential-backoff retries due to the third party API rate limiting problem that occurs during evaluation, not inherent to the architecture itself. Excluding those questions where an internal retry was performed, the mean/median latency becomes 36.84 s / 37.31 s, reported as the cost incurred by the full pipeline of the agentic system. Even with this value, Stage 3 appears to be around 8× slower than Stage 2 (4.43 s) and two orders of magnitude slower than the Baseline (0.39 s).

5.3. Evidence Sufficiency and Verification Behavior

The Verifier Agent’s claim-level evaluations result in an ESS average of 0.82 (median of 1.0, $\sigma = 0.33$) among the 120 analyzed answers, where the vector index is re-searched for the atomic claim independently without using the same retrieval context used to generate the answer. The complete ESS distribution where 88 out of 120 answers (73.3%) have achieved full claim support (ESS = 1.0), whereas 13 answers (10.8%) have been assigned ESS = 0.0 due to the fact that there was no independent supporting evidence or the system abstains itself.

5.4. Qualitative Analysis

Case 1 (Mis-attributed Literary Movement): In the case of the question “In which literary movement did the writer that featured in Alicia Ramsey’s historical play belong to?” (gold answer: *Romantic*), the first version of the Reasoner answered the question by stating that the writer was a leading figure in *Modernismo*, a semantically related but factually incorrect literary movement. The Verifier took the statement “the writer was a leading figure in *Modernismo*” as the claim and ran an independent retrieval on the corpus to see whether any passages supported the claim. Since there were no supporting passages for *Modernismo*, the claim was assigned an ESS of 0.0 and the Reasoner was asked to rework on the answer.

Case 2 (Mismatches between entity and answer type): For the query “In what 1995 British romantic comedy film did Richard E. Grant star?”, For which gold answer is *Jack and Sarah*, the system’s base generation step did not produce an answer that followed the required format and produced the name of the actor himself, *Richard E. Grant*. Upon parsing of the generated claim “Richard E. Grant is a 1995 British romantic comedy film”, the Verifier recognized it as being evidentially unsound because none of the passages contained information that characterized

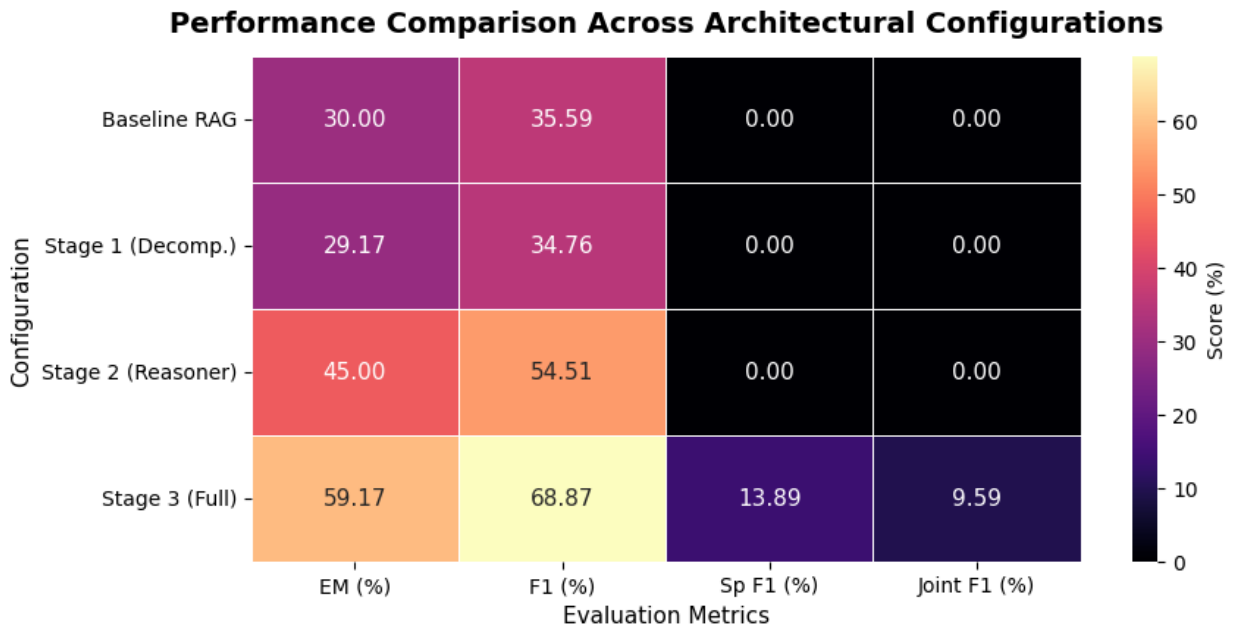


Fig. 2. Performance comparison across architectural configurations on HotpotQA ($N = 120$).

Richard E. Grant as a film. This is demonstrated by both examples through the expected result of decoupling the Verification stage from the Generation stage in each of them, the draft created by the Reasoner seemed fluent and plausible, however, the Verifier managed to find out that the specific fact claimed in the draft was not justified by the retrieved corpus.

6. Conclusion and future work

The proposed Agentic RAG framework improves question answering by dividing the process into four stages: planning, evidence retrieval, reasoning, and verification. Four agents Planner, Evidence Retrieval, Reasoner, and Verifier work together through LangGraph to generate accurate, evidence-based answers. The framework achieved strong results: the Planner reached 100% accuracy in question understanding and 87.2% in question decomposition, while the Retrieval Agent achieved 90% top-1 and 100% top-3/top-5 retrieval accuracy. Adding reasoning and verification further improved answer quality. However, the framework introduces additional processing time and depends on effective question decomposition, retrieval and reasoning. Its evaluation was limited to the HotpotQA dataset, so its effectiveness in real-world and diverse domains remains uncertain. Future work should focus on improving speed, retrieval and verification, testing across diverse datasets, and optimizing the balance between accuracy, reliability, computational cost, and response time. Overall, the framework supports trustworthy question answering by grounding responses in retrieved evidence and independently verifying answer claims.

Source code and Data Availability

The HotpotQA dataset used in this study is publicly available at https://huggingface.co/datasets/hotpotqa/hotpot_qa, and the implementation is available at <https://github.com/Jish-k/MultiAgent-enterprise-RAG>.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [3] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [4] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in *Association for Computational Linguistics (ACL)*, 2023.
- [5] L. Lahre, P. Singh, and N. K. Nagwani, “Integration of retrieval augmented generation (rag) with small language models (slms) for development of optimized agentic ai chatbot,” *Procedia Computer Science*, vol. 283, pp. 2243–2250, 2026.
- [6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [7] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [8] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and D. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [10] N. Shinn, F. Cassano, E. Berman, E. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders et al., “WebGPT: Browser-assisted question-answering with human feedback,” arXiv preprint arXiv:2112.09332, 2021.
- [13] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv preprint arXiv:2308.08155, 2023.
- [15] “Conversations with context: A modular transformer framework for understanding and generating natural language,” *Procedia Computer Science*, vol. 283, pp. 4426–4438, 2026.
- [16] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” arXiv preprint arXiv:2401.15884, 2024.
- [17] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [18] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, “Active retrieval augmented generation,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [19] “Medkg-llm: A framework for knowledge graph-augmented biomedical language models with explainable reasoning,” *Procedia Computer Science*, vol. 283, pp. 3842–3851, 2026.
- [20] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” in *Association for Computational Linguistics (ACL)*, 2023.
- [21] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive abstractive processing for tree-organized retrieval,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Y. Yu, W. Ping, Z. Liu, B. Wang, J. You, C. Zhang, M. Shoenybi, and C. Catanzaro, “RankRAG: Unifying context ranking with retrieval-augmented generation in LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [23] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A GraphRAG approach to query-focused summarization,” arXiv preprint arXiv:2404.16130, 2024.
- [24] S. U. Ahmed and N. Ahmad, “A rhetorical role relatedness (rrr) framework for legal case brief generation,” *Procedia Computer Science*, vol. 258, pp. 3333–3344, 2025.
- [25] V. G. Biju, B. Babu, A. Asghar, B. R. P. Boppuru, and V. Reddy, “From text to action: Nlp techniques for washing machine manual processing,” *Procedia Computer Science*, vol. 235, pp. 1903–1919, 2024.
- [26] A. Saxena, A. Santhanavijayan, and H. K. Shakya, “Fixing poorly written questions and classifying their difficulty with distilbert, albert, cnn, and explainable ai,” *Procedia Computer Science*, vol. 258, pp. 2130–2139, 2025.
- [27] M. A. Sayeed, D. Gupta, and V. Kanjirangat, “Engineering text-to-text generation language models as discriminative classifiers for accurate answer detection,” *Procedia Computer Science*, vol. 258, pp. 2930–2947, 2025.